

# Web API - Testing

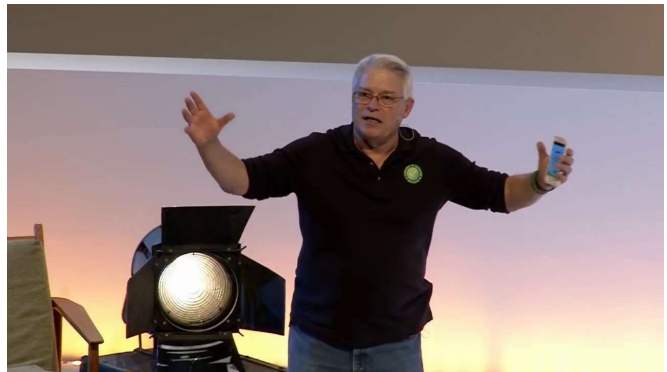


## Was ist Testing?

- Was verstehen Sie unter Testing?
- Welchen Nutzen sollte das Testen bringen?

Tests ermöglichen das Messen der Funktionalität und der Qualität an den Anforderungen und ermöglicht das Erkennen von Softwarefehlern.

### Clean Code - Uncle Bob / Lesson 4



<https://www.youtube.com/watch?v=58jGpV2Cg50>

## Welche Vorteile bringt Testing?

- Messen der Qualität
- Überwachen des Entwicklungsfortschritts
- Gewährleisten der Funktionalität auch nach langer Entwicklungsphase / Refactoring
- Motivation von Entwicklern
- Das „Alles ist Grün“ Gefühl, mit den Entwickler nach Hause gehen 😊

Weniger Bugs

Dokumentation

Wartbarer Code

Weniger Kosten als Integrationtesting, Functiontesting, Systemtesting

## Welche Nachteile hat Testing?

- Bei Einführung von z.B. TDD kann es zu einem höheren Einarbeitungsaufwand kommen
- Starker Wille des Entwicklers ist gefragt
- Unordentliche Struktur durch zu viele Tests
- Relation von Entwicklungszeit zu Testzeit kann aus dem Gleichgewicht kommen.

## Was sind Unit Tests?

- Test der in einer .NET Sprache geschrieben ist und einzelne Einheiten (Methoden) testen soll.

### Welchen Nutzen haben Unit Tests?

- Frühzeitiges Erkennen von Fehlern in Methoden
- Sicherheit nach dem Refactoring
- Sicherheit bei Updates / Bugfixing, dass noch alles funktioniert
- Aufzeichnung des Projektfortschritts

## Unit Test Frameworks in .NET



MSTest



Nunit



xUnit.net

## AAA Prinzip

- **Arrange**

- Benötigte Instanzen und Objekte werden erstellt
- Benötigte Hilfsfunktionen werden ausgeführt

- **Action**

- Eigentliches Testszenario (Methode) wird mit den in Arrange erzeugten Bedingungen erstellt

- **Assert**

- Ergebnisse der Methode werden mit den zu erwartenden Wert abgeglichen

## AAA Prinzip

```
[TestMethod]
public void Sum_3and4_returns7()
{
    //Arrange
    var calc = new Calc();

    //Act
    var result = calc.Sum(3, 4);

    //Assert
    Assert.AreEqual(7, result);
}
```



## Organisation von Unit Tests

- Erstellen eigener Assemblies
- Organisation in Verzeichnissen
- Pro Klasse eine Testklasse
  - Ordner für Klasse und per Methode eine TestClass

## Benennung von Unit Tests

```
[TestMethod]
public void Sum_3and4_returns7()
{
    //...
}
```

- Sum = Name der zu testenden Methode
- 3and4 = Szenario
- returns7 = Erwartetes Ergebnis

## Assert Basics

```
Assert.AreEqual("Peter", user.Name);
```

```
Assert.IsNotNull(user);
```

```
Assert.Fail();
```

```
Assert.Inconclusive();
```

```
Assert.Throws<CustomException>(new TestDelegate(user.Save));
```

Fluent-API Stil kann nur Nunit

## Weitere Assert-Klassen

- **StringAssert** 

```
StringAssert.Contains(user.Name, "Pete");
```

- **CollectionAssert** 

```
CollectionAssert.Contains(userList, user);
```

```
CollectionAssert.DoesNotContain(userList, user);
```

- **FileAssert** 

```
FileAssert.AreEqual("c:\\text1.txt", "c:\\text2.txt");
```

- **DictionaryAssert** 

```
DirectoryAssert.IsEmpty("c:\\temp");
```

## Test Lifecycle

```
[TestClass]
public class UnitTestClass
{
    //einmalig vor dem Testlauf
    [AssemblyInitialize]
    public static void InitAssembly(TestContext context)
    { }

    //einmalig für diese Klasse vor den Tests
    [ClassInitialize]
    public static void InitClass(TestContext context)
    { }

    //vor jedem Test dieser Klasse
    [TestInitialize]
    public void InitTest()
    { }

    //nach jedem Test dieser Klasse
    [TestCleanup]
    public void CleanTest()
    { }

    //einmalig für diese Klasse nach den Tests
    [ClassCleanup]
    public static void CleanupClass()
    { }

    //einmalig nach dem Testlauf
    [AssemblyCleanup]
    public static void CleanupAssembly()
    { }
}
```

## Was macht einen guten Unit Test aus?

- Testet genau eine „Unit“ (Methode)
- Isoliert -> keine Abhängigkeiten zu Filesystem, Datenbank usw. -> „In-Memory-Test“
- Läuft schnell
- Konsistent -> beliebig oft wiederholbar mit gleichem Ergebnis
- Reihenfolge egal -> keine Abhängigkeit zu anderen Unit Tests
- Einfach zu lesen

Alle Unit Tests ✓



## Was sind Integrationstests?

- Hat Abhängigkeit z.B. zu einem anderen System oder Filesystem, Datenbank, Webservice...
- ~~Am besten von den Unittests trennen (eigene Testassembly)~~
- Läuft eventuell nur in einer bestimmten Umgebung (Konfiguration, Datenbankinhalte)



## Einbeziehen der Datenbank (oder nicht)

- [SQL Server Express LocalDB](#) als performante "Single-User"-Datenbank
  - Realistischere Tests durch Nutzung von Tabellenstrukturen und Daten
- SQLite (In-Memory-Modus) als Datenbank-Fake
  - LINQ-Abfragen verhalten sich womöglich anders als mit SQL Server
- Gegen [Repository-Pattern](#) testen
  - Business Logik ist von Datenzugriff völlig getrennt, d. h. EF Core wird im Test-Stack nicht berücksichtigt
- **Nicht empfohlen** (Abfragen können sich anders verhalten oder fehl schlagen)
  - EF Core In-Memory-Provider als Datenbank-Fake
  - Mocking oder Stubbing von DbContext und DbSet

<https://learn.microsoft.com/en-us/ef/core/testing/choosing-a-testing-strategy>

17

<https://learn.microsoft.com/de-de/ef/core/testing/>

<https://learn.microsoft.com/en-us/ef/core/testing/choosing-a-testing-strategy>